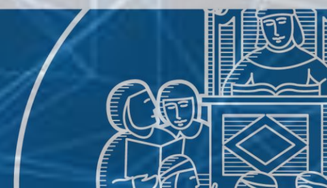
ROBOT PROGRAMMING FRAMEWORKS AND IOT PLATFORMS

Scuola Superiore Sant'Anna

Anno Accademico 2025/2026

Gastone Ciuti

IMT SCUOLA ALTI STUDI LUCCA



Organization

11/11/2025 Introduction and Communication Protocols (1/2) (3 hours – SLOT 1)

17/11/2025 Communication Protocols (2/2) (3 hours – SLOT 1)

18/11/2025 IoT Platforms and Main Features (3 hours – SLOT 2)

24/11/2025 Set-up of Particle Development Boards & Basic Functions (2 hours)

- Web-based programming, cloud functions and examples

25/11/2025 How to Program an IoT System: Functions and Coding (1/3) (3 hours – SLOT 3)

- Other Programming Environments, *i.e.* Particle Workbench based on Visual Studio Code

01/12/2025 How to Program an IoT System: Functions and Coding (2/3) (2 hours – SLOT 3)

- Advanced Programming Functionalities for IoT Platforms and Use Cases

02/12/2025 How to Program an IoT System: Functions and Coding (3/3) (3 hours – SLOT 3)

- Integration with Web Services, *i.e.* IFTTT (if this than that)

09/12/2025 Examples with IoT Systems and Exercises (3 hours – SLOT 4)

Set-up of Particle Development Boards & Basic Functions (25.11.2025) - SLOT 2

Gastone Ciuti

(1) Set up your ARGON

https://setup.particle.io/
MAIL: drims2@gmail.com
PASS: summerschooldrims2

ARGON in our class (please compile it)

Device name	Type	ID	Serial Number	Person
DOCENTE	ARGON	e00fce68f9a18538b172e40a		
DEV1	ARGON			
DEV2	ARGON			
DEV3	ARGON			
DEV4	ARGON			
DEV5	ARGON			
DEV6	ARGON			
DEV7	ARGON			

Where to find sensors and peripherals?

- **Particle store: https://store.particle.io/products/**
- **Seedstudio website: https://www.seeedstudio.com/**
- **Robot Italy: https://robot-italy.com/**
- **RS Components: https://it.rs-online.com/web/ | Farnell: https://it.farnell.com/ | DigiKey (link) | Mouser Electronics (link)**
- **Adafruit: https://www.adafruit.com/**

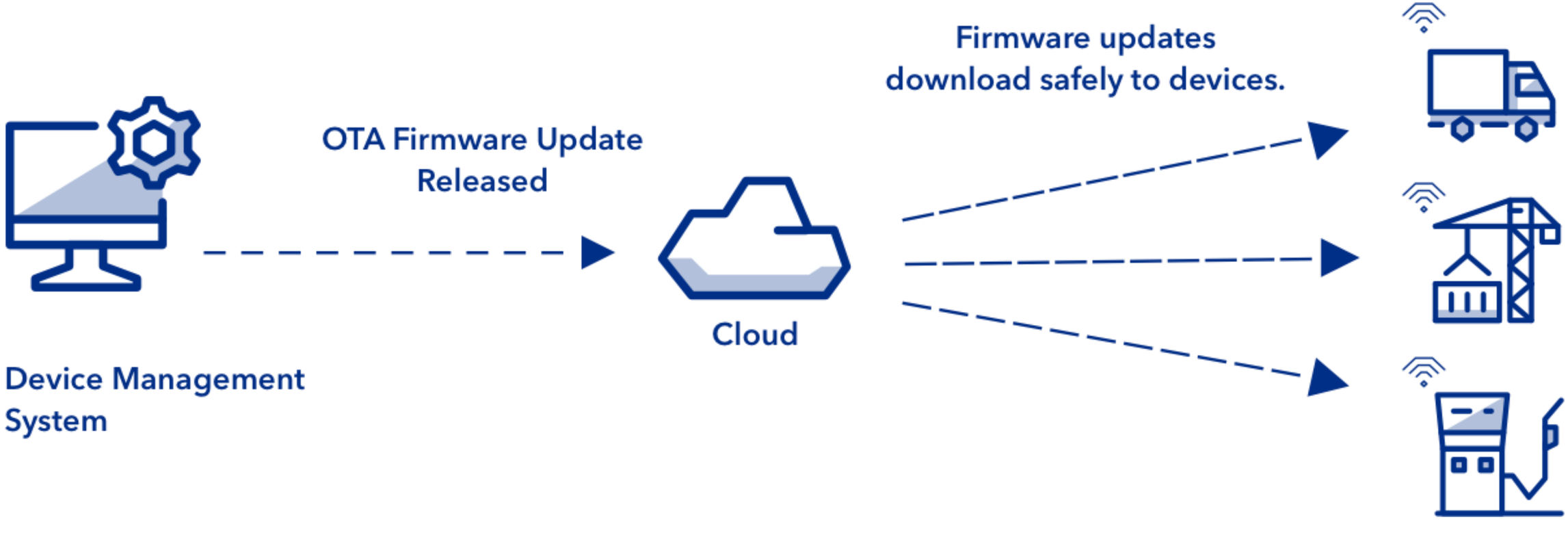
Where to download the Particle APP for Android?

- **Aptoide store: https://it.aptoide.com/**
- **App Particle for Android: http://particle-industries-inc-particle.en.aptoide.com/**

(1) Open the Web IDE

To program your Argon, open a new browser tab and go to the Web IDE. You will see a layout like the image below.

OTA firmware is delivered to remote devices using a device management system



Some basic functions and more, to know [link]

- **pinMode()** [link]
- **getPinMode(pin)** [link]
- **digitalWrite()** [link]
- **digitalRead()** [link]
- **analogWrite() (PWM)** [link]
- **analogRead() (ADC)** [link]
- **battery voltage** [link]
- **serial communication** [link]
- **SPI** [link] & **I2C** [link]
- **library search** [link]
- **web ide exporter** [link]

Example #0 – E0: blink an integrated LED, with OTA programming

Source [other HW examples]: https://docs.particle.io/tutorials/hardware-projects/hardware-examples/

E1: connect button, light, US, buzzer and rotary/angle peripherals

Source [other HW examples]: https://docs.particle.io/tutorials/hardware-projects/hardware-examples/

ROTARY SENSORS

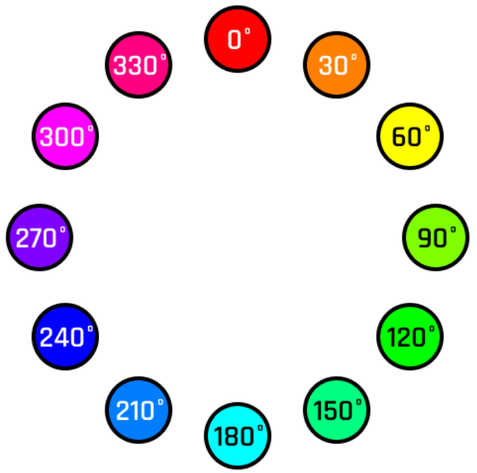
```
#define ADC_REF 3.3
#define GROVE_VCC 3.3
#define FULL_ANGLE 300

 int sensor_value = analogRead(ROTARY_ANGLE_SENSOR);
 voltage = (float)sensor_value*ADC_REF/4095; //12bit resolution
 float degrees = (voltage*FULL_ANGLE)/GROVE_VCC;
```

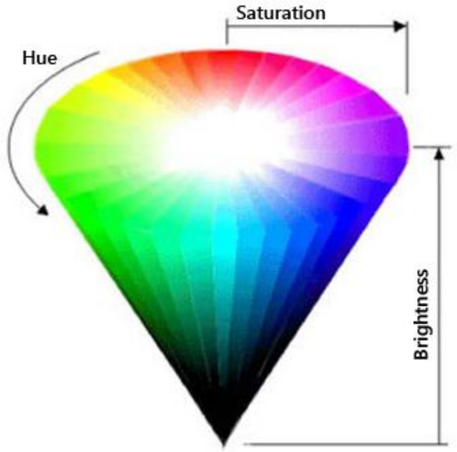
E2: connect RGB, FSR, vibro and Humidity&Temperature peripherals

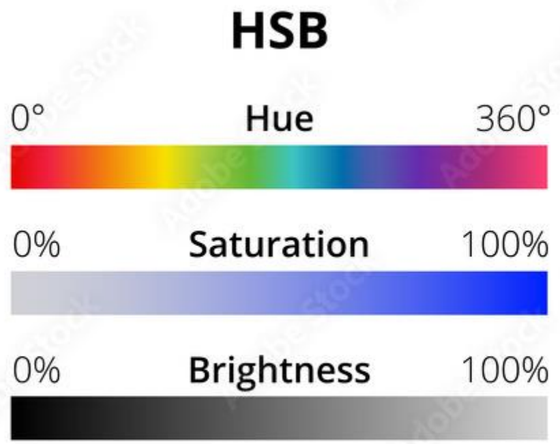
Source [other HW examples]: https://docs.particle.io/tutorials/hardware-projects/hardware-examples/

RGB LED (chainable)

HSL stands for hue, saturation, and lightness, and is often also called HLS. HSV stands for hue, saturation, and value, and is also often called **HSB (B for brightness)**. A third model, common in computer vision applications, is HSI, for hue, saturation, and intensity.





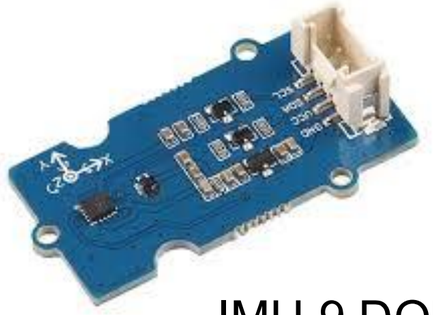


E3: air quality sensor

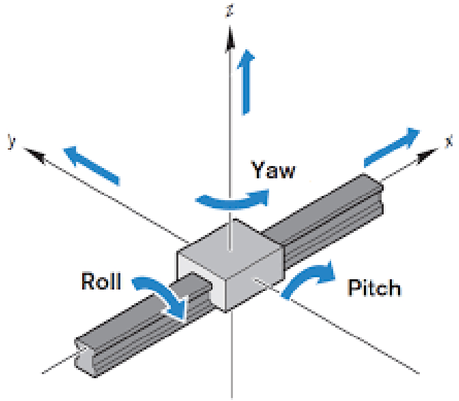
Source [other HW examples]: https://docs.particle.io/tutorials/hardware-projects/hardware-examples/

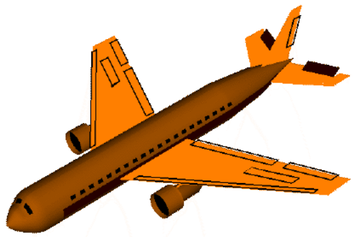
E4: accelerometer peripheral

Inertial Measurement Unit - IMU

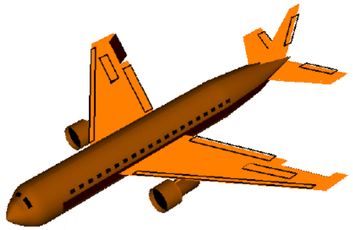


IMU 9 DOF
LCM20600 (3-axis gyroscope, 3-axis accelerometer)
AK09918 (3-axis magnetometer)

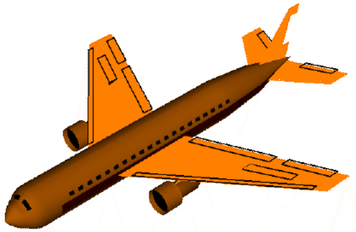




Pitch

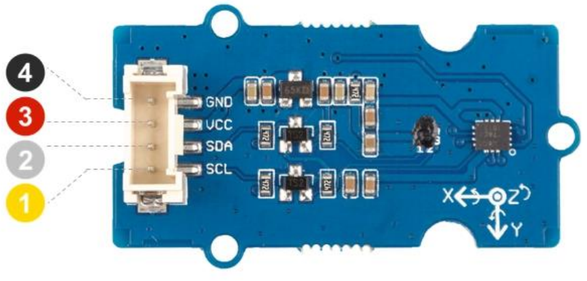


Roll

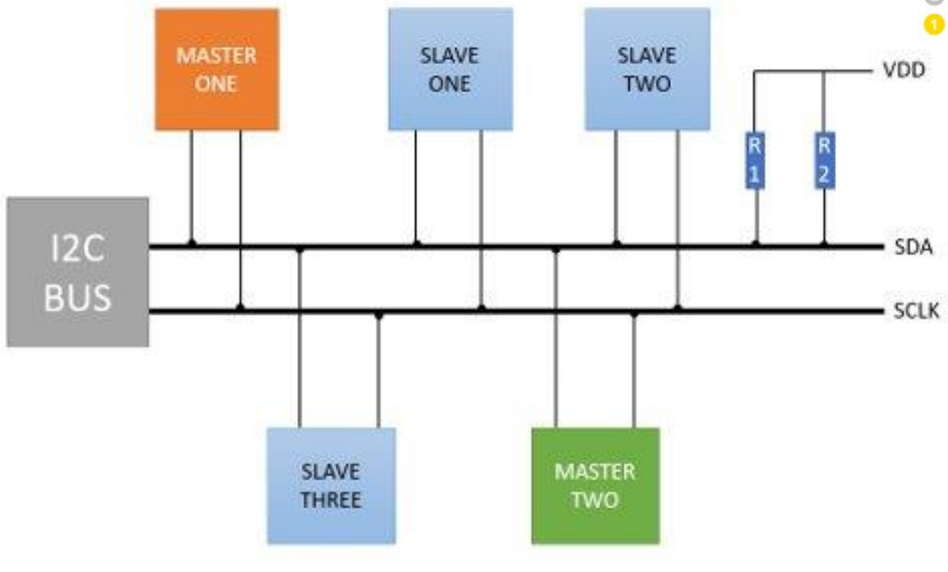


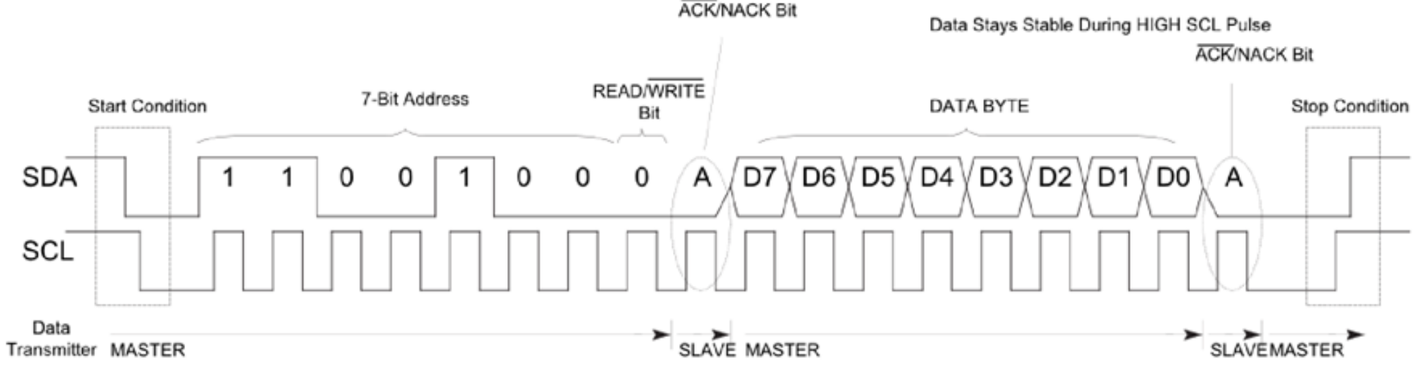
Yaw

Inertial Measurement Unit- IMU



4 GND: connect this module to the system GND
3 VCC: you can use 5V or 3.3V for this module
2 SDA: I²C serial data
1 SCL: I²C serial clock





Particle.function()

Expose a function through the Cloud so that it can be called with POST /v1/devices/{DEVICE_ID}/{FUNCTION}.

Particle.function allows code on the device to be run when requested from the cloud API. You typically do this when you want to control something on your device, say an LCD display or a buzzer, or control features in your firmware from the cloud.

```
// SYNTAX
bool success = Particle.function("funcKey", funcName);

// Cloud functions must return int and take one String
int funcName(String extra) {
  return 0;
}
```

Source [code]: https://docs.particle.io/cards/firmware/cloud-functions/particle-function/

E5: web connected led (particle function)

Particle.variable()

Expose a variable through the Cloud so that it can be called with GET /v1/devices/{DEVICE_ID}/{VARIABLE}. Returns a success value - true when the variable was registered.

Particle.variable registers a variable, so its value can be retrieved from the cloud in the future. You only call Particle.variable once per variable, typically passing in a global variable. You can change the value of the underlying global variable as often as you want; the value is only retrieved when requested, so simply changing the global variable does not use any data. You do not call Particle.variable when you change the value.

```
// EXAMPLE USAGE

bool flag = false;
int analogvalue = 0;
double tempC = 0;
char *message = "my name is particle";
String aString;

void setup()
{
  Particle.variable("flag", flag);
  Particle.variable("analogvalue", analogvalue);
  Particle.variable("temp", tempC);
  if (Particle.variable("mess", message) == false)
  {
      // variable not registered!
  }
  Particle.variable("mess2", aString);

  pinMode(A0, INPUT);
}

void loop()
{
  // Read the analog value of the sensor (TMP36)
  analogvalue = analogRead(A0);
  // Convert the reading into degree Celsius
  tempC = (((analogvalue * 3.3) / 4095) - 0.5) * 100;
  delay(200);
}
```

Source [code]: https://docs.particle.io/cards/firmware/cloud-functions/particle-variable/

E6: function (RGB-value) and variable (HT sensor)

Particle.publish()

Publish an event through the Particle Device Cloud that will be forwarded to all registered listeners, such as callbacks, subscribed streams of Server-Sent Events, and other devices listening via Particle.subscribe().

This feature allows the device to generate an event based on a condition. For example, you could connect a motion sensor to the device and have the device generate an event whenever motion is detected.

Particle.publish pushes the value out of the device at a time controlled by the device firmware. Particle.variable allows the value to be pulled from the device when requested from the cloud side.

```
// SYNTAX
Particle.publish(const char *eventName, PublishFlags flags);
Particle.publish(String eventName, PublishFlags flags);
```

Example #6: Publish (button, light sensor and potentiometer values)

Source [code]: https://docs.particle.io/reference/device-os/firmware/#particle-publish-

Particle.subscribe()

Subscribe to events published by devices.

This allows devices to talk to each other very easily. For example, one device could publish events when a motion sensor is triggered, and another could subscribe to these events and respond by sounding an alarm.

E7-9: publish (HT, rotary) and subscribe (RGB)

Source [code]: https://docs.particle.io/reference/device-os/firmware/#particle-subscribe-

References

- **Particle website:** https://www.particle.io/
- **Particle store:** https://store.particle.io/products/
- **Particle platform guides:** https://docs.particle.io/
- **Particle ARGON datasheet:** https://docs.particle.io/reference/datasheets/wi-fi/argon-datasheet/
- **Particle set-up:** https://setup.particle.io/
- **Particle «doctor»:** https://docs.particle.io/tools/doctor/
- **Particle reference manual:** https://docs.particle.io/reference/device-os/firmware/
- **Particle Web IDE:** https://build.particle.io/
- **Github repository:** https://github.com/particle-iot/argon
- **Seedstudio website:** https://www.seeedstudio.com/

gastone.ciuti@santannapisa.it